Data Visualization using PyPlot

→ Data

Data is one of the most valuable things today. All Big companies stores Data, which helps in diagnosis, focused services and many more

→ Data Visualization

It is basically graphical or visual representation of data and statistics using elements like charts, graphs and maps, etc.

- We can see patterns, trends, relations etc in a data

→ MatPlotLib Library

It provides many interfaces and functionality for 2D graphics in various forms. Basically it's a high quality Plotting library of Python. PyPlot is such a module in MatPlotlib. MatPloltib is a multi-Platform data visualization library built on NumPy arrays.

- To install MatPlotlib using pip type the following:
  Python -m pip install matPlotlib
- To import PyPlot :
  import matPlotlib.PyPlot as plt

→ PyPlot

MatPlotlib comes pre installed with anaconda
To run every command we need to write
matPlotlib.PyPlot.<command>
Better way : import matPlotlib.PyPlot as plt
plt.plot (<x>, <y>) → shorter way

→ Numpy

- Numpy is a module of python that offers functions for fast mathematical computation on arrays
- Arrays is a named group of homogeneous elements It's very similar to lists. All elements are of same datatype, functionality is different

→ Creating Array

1)
```
import numpy as np
L = [1,2,3,4]
a = np.array(L)          # makes 'a' array from list 'L'
print(type(a))           # numpy.ndarray
print(a.shape)           # (1,)
print(a.itemsize)        # 4
print(a.dtype)           # dtype('int32')
```

2)
```
import numpy as np
        a = np.arange(<start>, <stop>, <step>, <dtype>)
⇒ a = np.arange(1,9,3,np.float32)
        a = np.linespace(<start>, <stop, <number of values to be generated>) → will generate elements at
⇒ a = np.arange(1,9,2)                                    equal intervals
                                  ↳ ends at 8
```

→ Some more numpy functions

Trigonometric function

- sin(x) → sine function, element wise
- cos(x) → cosine function, element wise
- tan(x) → tangent function, element wise

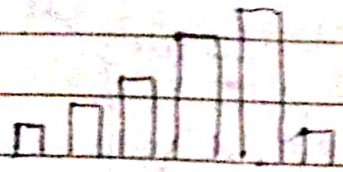
- arc sin (x) → inverse sine function, element-wise
- arc cos (x) → inverse cosine function, element-wise
- arc tan (x) → inverse tangent function, element-wise

Hyperbolic Functions
- sinh (x) → Hyperbolic sine function, element wise
- cosh (x) → Hyperbolic cosine function, element wise
- tanh (x) → Hyperbolic tangent function, element wise
- arc sinh (x) → Hyperbolic inverse sine function, element wise
- arc cosh (x) → Hyperbolic inverse cosine function, element wise
- arc tanh (x) → Hyperbolic inverse tangent function, element wise

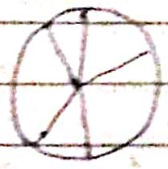
Rounding Functions
- round_(x, d) → Rounds an array to the give no. of decimals
- floor (x) → Returns the greatest integer function
- ceil (x) → Returns the smallest integer function
- trunc (x) → Returns the truncated value, element wise

Exp. & Log functions
- exp (x) → calculates the exponential of all elements
- exp2 (x) → calculates the 2**p for all p in the array, p are elements
- log (x) → calculates natural log, element wise
- log10 (x) → calculates base 10 log of the input array, element wise
- log2 (x) → base 2 log of x
- log1p (x) → Returns the natural log of one plus the input array, element wise

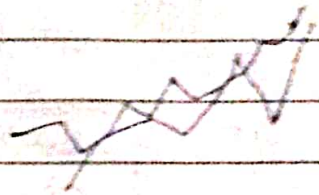
→ Charts



Bar charts



Pie charts



Line Chart

→ Line Chart

```
import numpy as nP
import matplotlib.pyplot as plt
a = nP.linspace (1, 5, 6)
b = nP.log (a)
plt.plot (a, b)
plt.xlabel ('This shows on x axis')
plt.ylabel ('This shows on y axis')
plt.show()
```

Multiple views

In case we want to plot legends in different views in the same window, we can use subplot() function.

plt.subPlot (numrows, numcols, fignum)

→ Changing styles

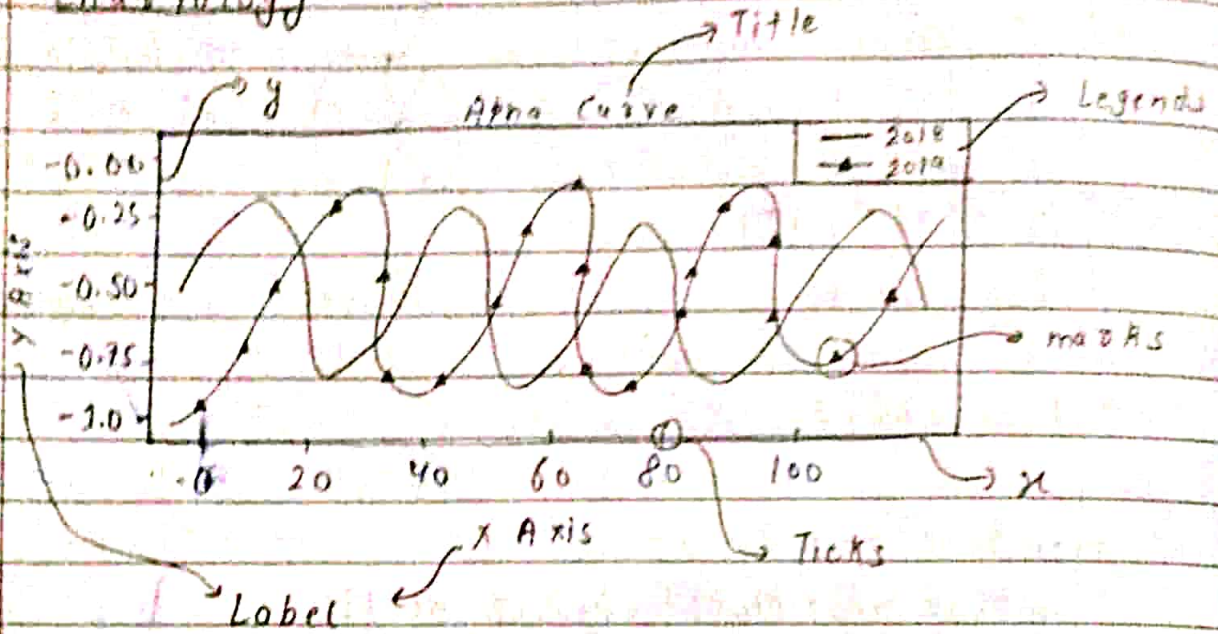
```
plt.plot (a-1, b-1, 'b')   → blue
plt.plot (a, b, 'r')       → red
```

} Multiple Charts

```
plt.plot (a+1, b+1, linewidth = 2, linestyle = 'dashed',
  marker = 'd', markersize = 4, markeredgecolor = 'red')
plt.plot (a+2, b+2, 'bd', linestyle = 'dashdot')
```

↳ blue and diamond

→ Chartology



→ Bar Graph

A bar graph is a very commonly used two dimensional data visualisation made up of rectangular bars, each for a specific category, with its length representing the value of that category

```
import matplotlib.pyplot as plt
a = [1,2,3,4,5]
b = [1,9,9,16,25]
plt.bar(a,b)
plt.xlabel('N')
plt.ylabel('N * N')
plt.show()
```

Default width of bars = 0.8 units

→ Styling Bars

```
a = ['Grapes','Banana','Apple','Guava']
plt.bar(a,b, width = [0.2,0.5,0.3,0.8])
plt.bar(a,b, color = ['g','y','b','black'])
```

Colors and width are applied in left to right order, but the bars are plotted in sorted order alphabetical

→ Histogram

```
V = [[5,25,45,20],[4,23,49,17],[6,22,47,19]]
X = np.arange(4)
plt.bar(X, V[0], color='b', width=0.25, label='a')
plt.bar(X+0.25, V[1], color='g', width=0.25,
    label='b')
plt.bar(X+0.50, V[2], color='r', width=0.25,
    label='c')
plt.legend(loc='upper left')
```

Color and width are applied in left to right order, but the bars are plotted in sorted order

→ Horizontal Bar Chart

```
import matplotlib.pyplot as plt
a = ['Grapes', 'Banana', 'Apple', 'Guava']
x = [4, 9, 2, 5]
plt.barh(a, x)
plt.show()
```

→ it is important to show

→ Pie Chart

A pie chart is a circular plot, divided into slices to show numerical proportion. Pie plot widely used in the business world

```
C = [4, 1, 8, 9]
D = [A1, B1, A2, B2]
plt.axis('equal')
plt.pie(C, labels=D)
```

→ for circular shape

Colors may differ

→ Styling Pie Charts

```
CLR = ['green', 'blue', 'red', 'yellow.']
E = [0,0,0,1,0,15]
plt.pie(C, colors = CLR, autopct = '%2.2f%%',
        explode = E)
```

Colors may differ — To show percentage

→ autopct = '% [FLAG] [WIDTH] . [PRECISION] type'

It's a special string which defines the structure of a string

- WIDTH specifies the min. no. of characters in string
- PRECISION is the no. of digits till which rounding off takes place after decimal
- type specifies the datatype, single % specifies it's a special string, That's why we use %%
- FLAG is the character we use to equalize the width. 'can be optional' default → -

Examples	value	auto percentage
'%3d'	24	-24
'%05i'	24	00024
'%03d%%'	24	024 %
'%6.1f'	24.2	--24.2
'%3.3f%%'	24.2	24.200 %

→ Customizing Chart

```
# plt.title('Student's data')
X = [0,1,2,3]
Y = [5,25,45,20]
plt.xticks([0,1,2,3],['a','b','c','d']) → tic ks
plt.bar(X,Y)
plt.title('Student\'s' data')
```

Page No.
Date

```
# plt.title('Student's data')
X = [0, 1, 2, 3]
Y = [5, 25, 45, 20]
plt.xlim(3.5, -0.5)      } limits
plt.ylim(-50, 50)
plt.title('Student's data')
pt.plot(X, X)
```

→ Some chars and markers

marker	description	marker	description
'.'	Point	'H'	hexagon 2
','	pixel	'1'	tri-down
'o'	circle	'2'	tri-up
'+'	plus	'3'	tri-left
'x'	x	'4'	tri-right
'D'	diamond	'v'	triangle-down
'd'	thin diamond	'^'	triangle-up
's'	square	'<'	triggle-left
'p'	pentagon	'>'	triangle-right
'*'	star	'|'	Vertical line
'h'	hexagon 1	'_'	Horizontal line

char	color
'b'	blue
'g'	green
'r'	red
'm'	magenta
'y'	yellow
'k'	black
'c'	cyan
'w'	white